

Use Case

Secure and **Rapid
Feature Delivery** for
a Decentralized
Finance (DeFi)
Startup





A fragmented DeFi startup's security model was transformed into a scalable DevSecOps framework by embedding automated smart contract testing, dependency scanning, and policy-as-code governance into the CI/CD pipeline—accelerating innovation while reducing protocol risk.

Overview

A fast-growing DeFi startup offered a suite of applications, including lending, yield farming, and token exchange protocols, built on the Ethereum Virtual Machine (EVM). The competitive nature of the DeFi space demanded continuous and rapid iteration, with new features and protocol upgrades being deployed weekly to capture market share. The core challenge was maintaining the highest standard of smart contract and application security while achieving this breakneck pace of development. A single security vulnerability could lead to the immediate loss of millions in user funds and irreversible reputational damage. The organization needed to implement a security-first development approach to accelerate feature delivery and respond to market demands without increasing the risk of catastrophic security flaws.

Challenges

As a FinTech provider operating on public blockchains, the startup faced risks unique to highly-regulated and high-value environments:

- **Speed vs. Security Trade-off:** Developers frequently bypassed thorough manual security reviews and formal audits to meet aggressive feature release timelines, leading to rushed code and unpatched vulnerabilities.
- **Late-Stage Security Audits:** Formal smart contract audits by external firms were conducted immediately before deployment. Critical bugs found late required significant, costly rework and delayed protocol launches.
- **Complex Attack Surface:** The application involved multiple attack vectors, including smart contract logic, backend APIs, front-end wallet integrations, and third-party oracle dependencies. Each new dependency increased the potential for a cascading failure.
- **Lack of Continuous Vulnerability Scanning:** Dependency updates (e.g., in node modules or libraries) and third-party protocol integrations were not continuously monitored for known vulnerabilities, leaving the application exposed to supply chain attacks.

- **Manual Compliance and Proof-of-Solvency:** Demonstrating protocol integrity, proof-of-reserves, and compliance with emerging regulatory requirements (e.g., AML/KYC for non-custodial services) was a fragmented, manual process that consumed significant engineering hours.
- **Inconsistent Deployment Integrity:** Managing deployment keys, multisig wallet policies, and secure Continuous Integration/Continuous Delivery (CI/CD) pipelines lacked standardization. This increased the risk of unauthorized or tampered code being deployed to the mainnet.
- **Siloed Teams and Knowledge Gaps:** Smart contract engineers, backend developers, and security experts operated independently. Security was often viewed as a gate to pass, rather than a shared, continuous responsibility across the development lifecycle.

Solutions

A tailored DevSecOps transformation was implemented, integrating security tooling and practices across the entire smart contract and application development lifecycle:

- **Security Integrated into CI/CD Pipelines:** Automated Static Application Security Testing (SAST) and dynamic analysis tools specialized for smart contracts (e.g., formal verification) were embedded directly into the Git workflow, running checks on every pull request.
- **Automated Dependency and Secret Management:** Automated scanning for vulnerable third-party dependencies and open-source libraries was implemented. Secure secret management and rotation for all API keys, database credentials, and deployment private keys were standardized.
- **Standardized Deployment Governance:** A policy-as-code approach ensured that only peer-reviewed and automatically scanned code could be deployed. All mainnet deployments required multisig approval and were executed via immutable CI/CD pipelines.



CYBERSEC GARD

Solutions

- **Continuous Security and Compliance Monitoring:** Security controls, logging, and policy enforcement aligned with financial compliance best practices (e.g., SOC 2 readiness) were embedded. On-chain monitoring tools provided immediate alerts on suspicious protocol behavior.
- **Shift-Left Security Training:** Developers received practical, continuous training on common smart contract exploits (e.g., reentrancy, integer overflow) and secure API design, enabling them to prevent flaws earlier.
- **Automated Audit Evidence:** Every commit and deployment automatically generated a detailed record of all security checks, code changes, and audit logs, making compliance evidence generation instantaneous and transparent.

Results

Before:

- Unpredictable and risky feature release cycles due to manual security bottlenecks.
- Critical vulnerabilities discovered late, leading to emergency mainnet patches and protocol downtime.
- Exposure to known vulnerabilities in third-party libraries.
- Manual, time-consuming compliance and proof-of-solvency evidence gathering.
- Inconsistent security practices for key and secrets management.
- Security team viewed as a roadblock, not a partner.

After:

- **40% faster time-to-market** for new features, safely accelerating iteration.
- **80% earlier detection of smart contract vulnerabilities**, reducing the need for costly external audits and major reworks.
- Continuous compliance readiness with automated audit trails and security evidence.
- Standardized, secure deployment and governance procedures for all protocol upgrades.
- Improved collaboration and a shared "security-first" culture among all engineering teams.
- Increased developer confidence, leading to more innovative and reliable protocol development.